「英眼书」构建与调试方法论

—一个背单词 Android 应用的工程实践复盘

项目：英眼书（com.kimi.cet4words） 引擎：Godot 4.7.1 版本：v0.9.11（versionCode 24）
日期：2026 年 10 月 12 日

这份文档回答两个问题：我在构建「英眼书」这款应用时遵循什么样的设计思路；在修改和调试它时遵循什么样的方法论。文中所有案例都来自真实的开发过程——从 AI 页面卡顿的修复，到文件选择器静默失败的根因排查，再到低配手机的性能适配。这些不是抽象原则，而是被逐一验证过的工程决策。

一、构建软件的思路

1.1 先分层，再写代码

动手写第一行代码之前，先确定架构。整个应用被划分为五个层次，每层只与相邻层对话，任何一层的实现可以被替换而不波及其他层：

层次	职责	对应实现
外壳层	生命周期管理、全局状态、分辨率与安全区适配	main.gd / autoload 单例
视图层	首页、学习、复习、AI、词库、设置六个页面的布局与交互	scenes/ 目录下的页面脚本
组件层	可复用控件：底部导航、单词卡片、弹窗、进度环	components/ 组件库
数据层	词库解析、学习进度存储、艾宾浩斯复习计划	data/ 数据模块
服务层	DeepSeek 对话、批量近义词生成、本地文件导入	services/ 服务模块

分层带来的直接收益在调试阶段体现得最清楚：文件导入失败时，能立刻判断问题在视图层（按钮事件）还是服务层（系统对话框调用），排查范围从整个工程缩小到一个函数。

1.2 防御式设计：用户数据是不可再生的

代码可以重写，用户积累的学习记录丢了就是丢了。因此所有写操作遵循同一条流水线：**校验 → 确认 → 备份 → 回滚**。以本地词库导入为例：解析器先对文件试算，弹出「共识别 N 个单词」的提示，用户确认后才写入；写入前自动备份旧词库，任何一步失败都回滚到导入前的状态

。宁可多一次确认，不可丢一条数据。

1.3 把「误触」当成功能需求来设计

触屏设备上，滑动和点击共用同一条事件通道——手指在屏幕上滑动时，极易在经过按钮的瞬间触发点击。这不是用户不小心，而是设计的失职。解决方案是 **28 像素触摸死区**：只有按下与松开两点间位移小于 28px 才判定为点击，否则视为滑动。这是 Godot 社区针对引擎 issue #16318 的标准解法，我把它作为全局规范写进了外壳层，所有按钮统一走这条判定链路。

1.4 性能预算化：不靠「感觉流畅」

性能问题用数字管理，不用体感管理。三条硬性预算：

- **每帧 6ms 渲染预算**：AI 聊天页的消息列表按批次实例化，每帧最多渲染一批，超出预算就推迟到下一帧，配合「代次作废」机制防止旧帧的滞后渲染堆积；
- **画质分级**：CPU 核心数 ⩽ 4 的设备自动进入低配模式，关闭投影与粒子特效，同时提供手动开关；高配与低配都锁定 60fps 目标；
- **覆盖底线**：GLES3 渲染器 + armv7/arm64 双架构 + minSdk 21，保证老旧低配机型可安装、可运行。

1.5 可测试性：先自证，再交付

每一次改动之后、交付之前，固定跑一套验证流程：无头模式启动工程确认零报错，运行 23 项核心功能测试和 14 项导入解析测试；涉及交互的改动再跑触控遍历脚本——它模拟真实手指事件对界面上每一个可点击元素逐一触发，84 次点击零失败才放行。测试不是开发的尾声，而是交付的定义：没有通过验证的改动，等于没有改。

1.6 开放数据与版本纪律

词库以纯文本形式开放，用户可以自己维护和导入；版本号遵守与用户的约定——在未宣布「正式版」之前，一律按 0.9.x 顺延（当前 v0.9.11），1.0 这个号码只留给用户亲口确认的那一刻。版本号是承诺，不是计数器。

二、修改与调试的思路

2.1 六步调试流程

所有缺陷修复都走同一条路径，无一例外：

步骤	要点	本次实践中的例子
1. 听懂描述	用户描述的是现象，不是根因。先把口语翻译成可观测的指标。	「打开 AI 会卡顿」→ 首帧渲染耗时、消息数量、实例化方式
2. 稳定复现	复现不了的问题不动手修，先补日志和观测点。	在低端机配置上复现滑动掉帧后才定位
3. 读代码定位	不猜。顺着调用链读到真正出问题的那一行。	卡顿根因：聊天页一次性实例化了全部历史消息
4. 外部证据	搜 GitHub issue 与官方文档；搜索「假设」而非预设的答案。	文件对话框静默失败由 issue #115983 / #116335 证实
5. 最小修复	改动面最小，但同类问题一次性修完。	封装 UI.file_dialog()，四个对话框调用点统一替换
6. 回归 + 记录	全量测试通过后，把根因写进开发记录的问题档案。	问题档案已累计 25 条，每条含现象、根因、解法

2.2 典型问题的根因对照

下表是本轮开发中六个代表性问题的完整链路。它们共同的教训是：**现象与根因之间几乎从不是直线**——「按钮没反应」的根因藏在一个被忽略的返回值里，「格式太乱」的根因是解析器对输入格式做了太强的假设。

用户看到的现象	真正的根因	最终的解法
打开 AI 页面时手机卡顿	进入页面时一次性实例化全部聊天记录	分批渲染 + 每帧 6ms 预算 + 代次作废
本地导入按钮按下无反应	忽略了 file_dialog_show 的返回值，失败时静默退出	统一封装 UI.file_dialog()，检查错误码并对全文件类型兜底
滑动列表时误触按钮	点击与滑动共用事件通道，未做位移区分	28px 触摸死区，位移超阈值一律判为滑动
AI 输入框被输入法遮挡	布局未感知软键盘弹出后的可视区域变化	用 virtual_keyboard_get_height 补高度差值，输入框上移
导入脚本报「格式太乱」	解析器假设输入是规整格式，遇到混排即失败	宽松解析：多模式分行、脏字符清洗、跳过无法识别的行

用户看到的现象	真正的根因	最终的解法
AI 长回复被截断	回复超过 token 上限后无续传机制	改用 response_format: json_object 结构化输出，近义词生成 598/605 词成功

2.3 三条调试纪律

- **根因不明，绝不动手。**靠猜修好的问题会以另一种形式回来。文件选择器问题如果当时只「再点一次试试」，静默失败会一直潜伏；
- **修一个点，查一个面。**发现某处忽略了返回值，就排查全部同类调用——四个文件对话框在同一次修复中统一封装，杜绝按下葫芦浮起瓢；
- **证据优先于经验。**训练知识有截止日期，而 issue 追踪和官方文档是活的。遇到引擎行为存疑时，先搜「这个假设是否成立」，再决定改哪行代码。

三、质量保障：交付前的固定动作

所有方法论最终落在一张清单上。每个版本导出 APK 之前，必须依次完成：

- 无头模式启动工程，控制台零报错；
- test_runner 23 项核心功能测试全部通过；
- test_import_parse 14 项导入解析测试全部通过；
- test_touch_all 触控遍历：84 次模拟手指点击，失败数为零；
- 导出 APK（GLES3、armv7 + arm64、minSdk 21）并在目标配置上实机验证；
- 更新开发记录：问题档案、测试矩阵、版本日志三项同步。

如果把这个方法论压缩成一句话，那就是：**听得懂、复现得了、找得到根、修得最小、验得全、记得住**。前两条决定改的东西对不对，中间两条决定改得稳不稳，最后两条决定同样的坑会不会再掉第二次。